

1、Langchain框架入门

- 1、什么是 LangChain
- 2、langchian 的发展历史
- 3、创建第一个 Agent
- 4、接入langsmith
 - 1、安装依赖包
 - 2、登录 langsmith
 - 3、创建秘钥 并保存
 - 4、配置环境变量

官方文档：https://python.langchain.com/docs/how_to/installation/

1、什么是 LangChain

LangChain是基于大型语言模型（LLM）应用程序的**开源框架**。它提供了一套工具和接口，简化了LLM 应用的开发流程，使开发者能够更高效地构建复杂的、数据感知的AI应用，LangChain 的出现填补了 LLM 工程化的空白，让开发者能快速构建生产级应用（如客服机器人、数据分析工具），而无需从零实现底层交互逻辑。

理解：LangChain ≈ 大模型的“编排中间件”，让你可以像用 FastApi 写 Web 一样，快速写出一个智能体系统。

- 官方文档：https://python.langchain.com/docs/how_to/installation/
- 中文文档：<https://docs.langchain.org.cn/>

基本环境安装

```
1 pip install langchain
2
3 pip install langchain_openai
```

2、langchian 的发展历史

鉴于 AI 领域不断变化的步伐，LangChain 也随时间演变。以下是 LangChain 多年来如何随着使用 LLM 构建应用的意义而变化的简要时间线

时间	版本 / 事件	描述
2022-10-24	v0.0.1 发布	LangChain 首次以 Python 包发布。包含两个核心组件：① LLM 抽象；② 链（Chain），用于构建如 RAG 的预定义步骤。名称来源于“Language + Chain”。
2022-12	第一个通用智能体	基于 ReAct（推理 + 行动）论文的通用 Agent 加入 LangChain。通过 LLM 生成表示工具调用的 JSON，并由框架解析后执行。
2023-01	ChatCompletions 支持	OpenAI 发布 ChatCompletions API，模型从字符串接口转为消息列表接口。LangChain 同步更新支持消息格式。
2023-01	JS 版本发布	LangChain 发布 JavaScript 版本，以满足前端/全栈开发者的 LLM 应用需求。
2023-02	LangChain Inc. 成立	LangChain 公司成立，使命为“让智能体无处不在”。开始构建围绕 LangChain 的完整生态。
2023-03	Function Calling 支持	OpenAI 发布函数调用（Function Calling），提供结构化的工具调用方式。LangChain 更新为优先使用函数调用，而不再依赖 JSON 解析。
2023-06	LangSmith 发布	LangSmith（闭源）发布，用于智能体可观测性与评估。LangChain 也随之更新为可与 LangSmith 深度集成。
2024-01	v0.1.0 发布	LangChain 首个非 0.0.x 稳定版本。行业进入生产化阶段，框架更重视 API 稳定性。
2024-02	LangGraph 开源	引入用于智能体编排的底层状态机框架，补足 LangChain 仅支持高级接口的不足。新增支持：流式、持久执行、短期记忆、人机协作等。
2024-06	700+ 集成	LangChain 生态超过 700 个集成。核心集成迁移到独立包或 langchain-community，使核心库更轻量。
2024-10	LangGraph 成为主流编排方式	构建 AI 应用的首选方式从 LangChain Chains/Agents 迁移到 LangGraph。大多数旧接口被标记为弃用，并提供迁移指南。LangGraph 仍提供一个高级 Agent 抽象，与早期 ReAct Agent 相兼容。

2025-04	消息格式支持多模态	模型 API 全面多模态化：支持输入文件、图像、视频。 LangChain 更新消息格式以标准化多模态输入。
2025-10-20	v1.0.0 发布	两大变化：① 所有 Chains 和 Agents 全部被统一为一个基于 LangGraph 的高级 Agent 抽象；旧版接口迁移到 <code>langchain-classic</code> ； ② 标准化消息内容格式，以兼容推理块、引用、服务器端工具调用等更复杂的输出格式。

3、创建第一个 Agent

Agent 介绍

智能体将语言模型与[工具](#)结合起来，创建能够推理任务、决定使用哪些工具并迭代寻找解决方案的系统。[create_agent](#) 提供了生产准备的代理实现。[LLM代理会循环运行工具以实现目标](#)。智能体运行直到满足停止条件——即模型输出最终输出或达到迭代极限

```

1  import os
2  import warnings
3  import dotenv
4  from langchain.chat_models import init_chat_model
5  from langchain.agents import create_agent
6  from langchain.tools import tool
7
8  # 忽略 pydantic 警告
9  warnings.filterwarnings("ignore", category=UserWarning, module="pydantic.*")
10
11 # 加载环境变量
12 dotenv.load_dotenv()
13
14 model = init_chat_model(
15     model=os.getenv("llm_model"),
16     model_provider="openai",
17     base_url=os.getenv("base_url"),
18     api_key=os.getenv("api_key"),
19 )
20
21 @tool
22 def get_weather(city: str) -> str:
23     """获取城市天气信息"""
24     # 这里可以接入真实的天气API
25     weather_data = {
26         "北京": "晴, 温度 25°C, 湿度 45%",
27         "上海": "多云, 温度 28°C, 湿度 60%",
28         "长沙": "晴转多云, 温度 30°C, 湿度 55%",
29         "深圳": "阵雨, 温度 32°C, 湿度 75%"
30     }
31     return weather_data.get(city, f"{city}的天气信息暂不可用")
32
33 @tool
34 def get_air_quality(city: str) -> str:
35     """获取空气质量信息"""
36     # 这里可以接入真实的空气质量API
37     air_quality_data = {
38         "北京": "PM2.5指数: 85, 空气质量: 良",
39         "上海": "PM2.5指数: 65, 空气质量: 良",
40         "长沙": "PM2.5指数: 80, 空气质量: 良",
41         "深圳": "PM2.5指数: 45, 空气质量: 优"
42     }
43     return air_quality_data.get(city, f"{city}的空气质量信息暂不可用")
44
45

```

```

46 agent = create_agent(
47     model=model,
48     tools=[get_weather, get_air_quality],
49     system_prompt="""你是一个专业的天气查询助手。请遵循以下规则：
50         1. 根据用户问题选择合适的工具查询天气和空气质量
51         2. 回答要简洁明了，包含具体数据
52         3. 如果用户没有指定城市，请主动询问
53         4. 可以提供一些出行建议""",
54 )
55
56 response = agent.invoke({
57     "messages": [
58         {"role": "user", "content": "长沙今天的天气怎么样? "}
59     ],
60 })
61 print(response['messages'][-1].content)

```

4、接入langsmith

LangSmith 提供开发、调试和部署 LLM 应用的工具。它帮助你在一个地方追踪请求、评估输出、测试提示并管理部署。LangSmith 不受框架限制，所以你可以和 LangChain 的开源库 [langchain](#) 和 [langgraph](#) 一起使用，也可以不使用。先在本地进行原型开发，然后通过集成监控和评估进入生产，构建更可靠的人工智能系统。

- 文档官网地址:<https://docs.langchain.com/langsmith/home>

1



创建一个账户

smith.langchain.com 注册（无需信用卡）。你可以用Google、GitHub或电子邮件登录。



创建一个API密钥

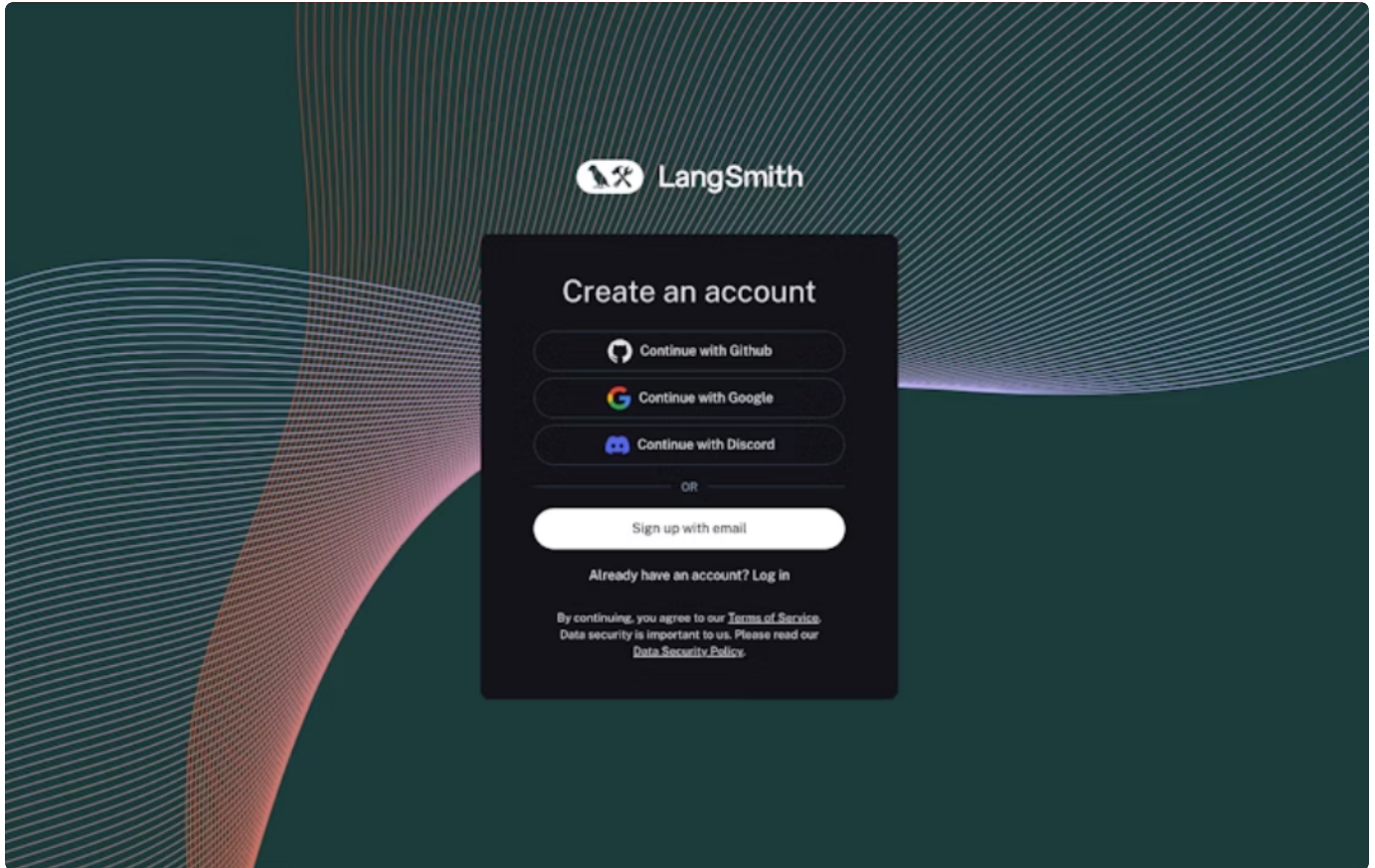
进入[设置页面](#)→ API 密钥→创建 API 密钥。复制密钥并安全保存。

1、安装依赖包

```
1 pip install langsmith
```

2、登录 langsmith

langsmith 地址: <https://smith.langchain.com/>, 推荐使用 github 账号登录



3、创建秘钥 并保存

要用 LangSmith 记录追踪和运行评估, 你需要创建一个 API 密钥来认证你的请求。API 密钥可以作用域限定到一组[工作区](#), 也可以覆盖整个[组织](#)。要创建任一类型的API密钥:

1. 进入[设置页面](#), 滚动到**API密钥**部分。
2. 对于服务密钥, 请选择组织范围和工作空间范围键。如果键是工作区范围, 则必须指定工作区。企业用户还可以[为密钥分配特定角色](#), 从而调整其权限。
3. 设置密钥的过期; 选定天数后密钥将无法使用, 或者如果选择了天数, 密钥将永远无法使用。
4. 点击**创建API密钥**。

LangSmith

个人

设置资源标签

家

可观察性

追踪项目

0

监测

0

评估

数据集与实验

0

注册队列

0

提示工程

提示

0

编排

部署

0

部署

部署

0

代理

特工建造者

试用版

我们开始吧

欢迎来到朗史密斯

开始追踪

用预建的ReAct代理设置追踪

在Studio中调试和迭代

可视化你的图架构并实时测试

试试游乐场

测试不同的提示和模型

做个实验

衡量应用性能并确保可靠性

个人

身份证

可观察性

追踪项目

0

自定义仪表盘

0

名字	最近一次演出 (7D)	反馈 (7D)	得分计数 (7D)	错误率 (7D)	P50延迟 (7D)	P99延迟 (7D)
可观察性设置 发送追踪到 LangSmith, 创建一个新的追踪项目并 开始监控 可观察性设置						

展示5个最活跃的项目。

设置

文档

最新动态

联系销售

邀请

主题

个人

返回

组织

组织ID

个人

工作空间

成员与职责

API 密钥

OAuth 提供者

模型

共享

秘密

反馈标签

计费与使用

文档

最新动态

联系销售

邀请

主题

个人

mskij@qq.com

API 密钥

个人

服务

API 文档

+ API 密钥

密钥	到期	描述	诞生于	最后一次使用	行动
----	----	----	-----	--------	----

创建API密钥

描述

2503期上课演示

密钥类型

个人访问令牌
用于作为个人用户使用 API 进行身份验证。

服务密钥
与自动化和CI工作流程的服务主体绑定。

默认工作区

工作区1

当没有提供 X-Tenant-ID 头部时，该工作区将作为默认使用。

有效期

从不 30分 90d 1年 习惯

取消 创建API密钥

4、配置环境变量

在代码的.env 中配置 langsmith 的 API_KEY

```
1 LANGSMITH_TRACING=true
2 LANGSMITH_ENDPOINT=https://api.smith.langchain.com
3 LANGSMITH_API_KEY=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
4 LANGSMITH_PROJECT=2503上课演示
```